20.1

Leave your fears behind & you'll find, you could make it after all. (Insha Allah)

Explain the distinction between the written sequence and the time sequence of aninstruction.

=>

The operation of a computer, in executing a program, consists of a sequence of instruction cycles, with one machine instruction per cycle. This sequence of instruction cycles is not necessarily the same as the written sequence of instructions that make up the program, because of the existence of branching instructions. The actual execution of instructions follows a time sequence of instructions.

2

One more leap of faith, 1 more step, clench your teeth & take breath.

What is the relationship between instructions andmicro-operations?

=>

A micro-operation is an elementary CPU operation, performed during one clock pulse. An instruction consists of a sequence of micro-operations

3

This much I know it won't go to waste.

What is the overall function of a processor's control unit?

=>

The control unit of a processor performs two tasks:

(1) It causes the processor to execute micro-operations in the proper sequence, determined by the program being executed, and

(2) it generates the control signals that cause each

microoperation to be executed.

4

Outline athree-step process that leads to a characterization of the control unit.

=>

1. Define the basic elements of the processor. 2. Describe the micro-operations that the processor performs. 3. Determine the functions that the control unit must perform to cause the micro-operations to be performed.

5 Same as 3 it seems

What basic tasks does a control unit perform?

=>

Sequencing: The control unit causes the processor to step through a series of micro-operations in the proper sequence, based on the program being executed.

Execution: The control unit causes each micro-operation to be performed.

6

Provide a typical list of the inputs and outputs of a control unit.

=>

The inputs are:

Clock: This is how the control unit "keeps time." The control unit causes one micro-operation (or a set of simultaneous micro-operations) to be performed for each clock pulse. This is sometimes referred to as the processor cycle time, or the clock cycle time.

Instruction register: The opcode of the current instruction is

used to determine which micro-operations to perform during the execute cycle.
Flags: These are needed by the control unit to determine the status of the processor and the outcome of previous ALU operations.
Control signals from control bus: The control bus portion of the system bus provides signals to the control unit, such as interrupt signals and acknowledgments.

The outputs are:
Control signals within the processor: These are two types: those that cause data to be moved from one register to another, and those that activate specific ALU functions.
Control signals to control bus: These are also of two types: control signals to memory, and control signals to the I/O modules.

7

List three types of control signals.

=>
(1) Those that activate an ALU function. (2) those that activate a data path. (3)Those that are signals on the external system bus or other external interface

8

Briefly explain what is meant by a hardwired implementation of a control unit. =>
In a hardwired implementation, the control unit is essentially a combinatorial circuit. Its input logic signals are transformed into a set of output logic signals, which are the control signals.

Maths

(1)

Your ALU can add its two input registers, and it can logically complement the bits ofeither input register, but it cannot subtract. Numbers are to be stored in twos complement representation. List themicro-operations your control unit must perform tocause a subtraction

=> Instr. SUB R_1, X :

Subtracts contents of location X from Contents of reg R1, places the result in R1.

t_1: MAR ← (IR(address)) ← X

t_2: MBR ← Memory

t_3: MBR ← complement (MBR)

t_4: MBR ← Increment (MBR)

t_5: R1 ← R1 + (MBR)

(2)

Show themicro-operations and control signals in the same fashion as Table 20.1 forthe processor in Figure 20.5 for the following instructions:

- Load Accumulator → fetch already done.
- Store Accumulator
- Add to Accumulator
- AND to Accumulator
- Jump
- Jump ifAC = 0
- Complement Accumulator

=>

Micro OP: | Active Control signals

LOAD AC: t_1: MAR ← (IR(address)) — C_8

t_2: MBR ← Memory — C_5, C_R

t_3: AC ← MBR — C_{10}

STORE AC: t_1: MAR ← (IR(address)) — C_8

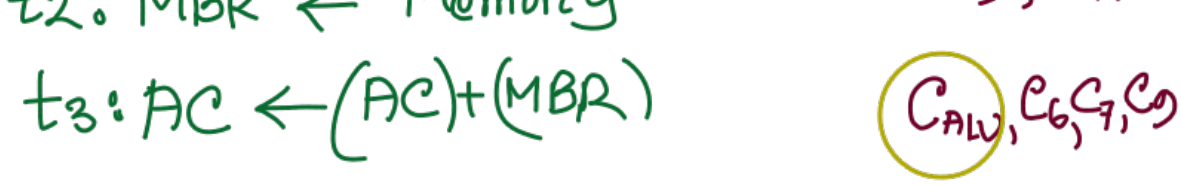
t_2: MBR ← AC — C_{11}

t_3: Memory ← MBR — C_{12}, C_W

(check ch.. gist 20.2 for details)

ADD AC: t_1: MAR ← (IR(address)) — C_8

t_2: MBR ← Memory — C_5, C_R



Note: There must be a delay between the activation of C8 and C9, and one or
more control signals must be sent to the ALU. All of this would be done during
one or more clock pulses, depending on control unit design

1. C8 Sets the Address (t1):
At time step t1, C8 is used to load the address field from the instruction register (IR) into the MAR (Memory Address Register).

This does not yet involve accessing memory — it's just setting up the address.

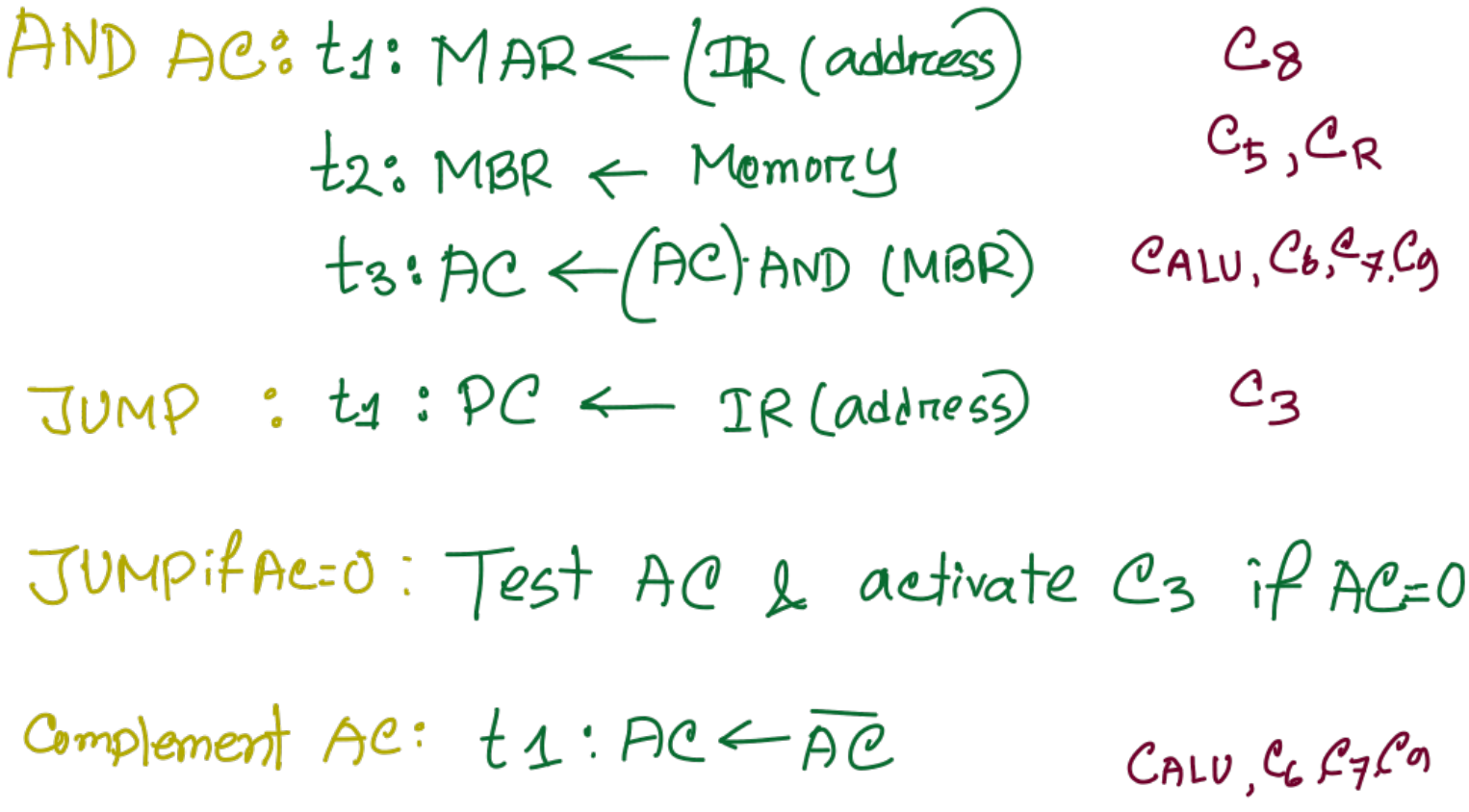
2. Memory Access Takes Time (t2):
At t2, we use control signals C5 and CR to read from memory at the address stored in MAR.

This is a relatively slow operation compared to register-to-register transfers — it requires waiting for memory response time.

Therefore, you cannot immediately proceed to ALU operations (like ADD) in the same clock cycle.

3. C9 Writes ALU Result to AC (t3):
C9 enables the output of the ALU to be stored into the AC.

But if you were to activate this too soon (before the MBR has received the operand from memory), the ALU would be adding invalid or incomplete data.



What Is the *Interrupt Cycle* For?

The **interrupt cycle** is **not** a normal part of instruction execution. It’s used only when **an external or internal interrupt signal is received** — such as:
- A hardware event (e.g., I/O device ready)
- An exception (e.g., divide by zero)
- A timer signal or system trap

Purpose of Interrupt Cycle:
- Save the current program state (typically `PC`) to memory
- Jump to a predefined **interrupt handler** address
- Later, return to the interrupted program (via some `RET`-like instruction)

What Do `JUMP` and `JUMP IF AC = 0` Do?

Let’s look at these two instructions:

1. JUMP
- Transfers control to another address **specified in the instruction**
- Micro-op:
  - `t1: PC ← IR(address)`
    - Control signal: `C3`

2. JUMP IF AC = 0
- **Conditionally** transfers control
- If AC is zero:
  - `PC ← IR(address)`
    - Only activate `C3` if condition is true

Key Point:
These instructions are just *normal control flow changes*. They do **not** involve saving the current state or dealing with asynchronous events. They are fully part of the **regular instruction execution cycle**.

⚠️ Why Not Interrupt Cycle?

Feature	JUMP / JUMP IF AC=0	Interrupt Cycle

-|
Triggered by	Instruction in the program	External/ internal signal
Purpose	Change PC to a target address	Save PC & jump to interrupt handler
Saves program state?	❌ No	✅ Yes (saves PC into memory)
Return address stored?	❌ No	✅ Yes (needed to resume execution)
Involves MBR/Memory Save	❌ No	✅ Yes (PC is stored at a known address)

✅ In Summary

- **JUMP** and **JUMP IF AC = 0** are standard program control instructions.
- They are fully handled by the **execution cycle**, not the **interrupt cycle**.
- The **interrupt cycle is reserved** for responding to exceptional or asynchronous events and must save the current PC, unlike JUMP instructions.



Assume that propagation delay along the bus and through the ALU of Figure 20.6 are20 and 100 ns, respectively. The time required for a register to copy data from the busis 10 ns. What is the time that must be allowed for
a. data from one register to another?
b. the program counter?

Given:

- **Propagation delay along the bus** = 20 ns
- **ALU propagation delay** = 100 ns
- **Register copy time from bus** = 10 ns

(a) Time required to transfer data from one register to another

This operation consists of:
- Data from source register placed on the bus → **bus propagation delay = 20 ns**
- Destination register copies data from the bus → **copy time = 10 ns**

Total time = 20 ns + 10 ns = 30 ns**

Answer (a): 30 ns

(b) Time required to increment the program counter (PC)

Incrementing PC typically involves **two micro-operations**:

Step 1: Z ← PC + 1**

- PC value placed on the bus → **20 ns**
- ALU adds 1 to the PC value → **100 ns**

- Result stored in temporary register Z → **10 ns**

Time for Step 1 = 20 + 100 + 10 = 130 ns**

Step 2: PC ← Z**

- Z places value on the bus → **20 ns**
- PC copies from the bus → **10 ns**

Time for Step 2 = 20 + 10 = 30 ns

Total time for PC increment = 130 ns + 30 ns = 160 ns**

Answer (b): 160 ns**



Write the sequence ofmicro-operations required for the bus structure of Figure 20.6to add a number to the AC when the number isa. b. c. immediate operand;direct-address operand;indirect-address operand.

=>

a) immediate: t_1: Y ← (IR(address))

t_2: Z ← (AC) + (Y)

t_3: AC ← Z

b) direct: t_1: MAR ← (IR(address))

t_2 : MBR ← Memory

t_3 : Y ← MBR

t_4 : Z ← (AC) + Y

t_5 : AC ← Z

c) Indirect: t_1 : MAR ← (IR (address))

t_2 : MBR ← Memory

t_3 : MAR ← (MBR)

t_4 : MBR ← Memory

t_5 : Y ← MBR

t_6 : Z ← (AC) + Y

t_7 : AC ← Z

20.5

A stack is implemented as shown in Figure 20.11 (see Appendix I for a discussion ofstacks). Show the sequence ofmicro-operations fora. popping;b. the stack

⇒ b → Assuming value to be pushed in R1.

a) Popping : t_1 : SP ← SP + 1

b) push : t_1 : SP ← SP − 1

MBR ← R1

t2 : MAR ← SP

t3 : Memory ← MBR

Below is the usual three-cycle micro-operation sequence for a **push** on the configuration of Fig 10.14a, assuming a **full, descending** stack and that the word to be pushed is in register **R1**. We break it into time-steps t1–t3, exactly as you sketched:

Time	Micro-operations	Comment
t1	`ALU ← SP − 1` `SP ← ALU`	decrement SP to reserve new slot
t2	`MBR ← R1` `MAR ← SP`	load the data into MBR, point MAR at the new top
t3	`M[MAR] ← MBR`	write the pushed value into memory

And for completeness, here's the complementary **pop** sequence (three cycles) that restores into **R1**:

Time	Micro-operations	Comment

t1	`MAR ← SP`	point MAR at current top
t2	`MBR ← M[MAR]` `R1 ← MBR`	read the word and deposit into R1
t3	`ALU ← SP + 1` `SP ← ALU`	increment SP to remove that slot

Key points:

- We assume a **full, descending** stack, so SP points at the last used slot; pushing decrements SP, popping increments it.
- **MBR** is the memory-buffer register, **MAR** the memory-address register.
- Each of t1, t2, t3 corresponds to one clock-cycle micro-operation group.